



by Joseph Michela

Relief from the bookkeeping of change management

Feature

Aug 13, 2026 • 7 mins

Many change management processes are necessarily heavyweight. But for changes that are reversible, there's an opportunity to lighten the load.



Credit: Dudarev Mikhail / Shutterstock

It's an idle Tuesday afternoon. You're in hour three of change management calls to get a CostCenter tag update approved for a production change. Only finops reporting is affected. The change itself will take two minutes of Terraform execution. Last week, you had a `DROP TABLE` action that destroyed 10TB of obsolete tracking data. It was the same hour of doc writing and three hours of change management review calls for both changes.

Most change management systems focus on generating coordination between teams, providing before-after evidence, and establishing a rollback path. These feed into the commonly articulated requirements like collision avoidance, regulatory compliance, scheduling, and auditability/traceability. These requirements cannot be hand-waved away. But perhaps the evidence to meet those requirements could be had for cheap(er)? Let's see if we can do better.

Ideally, we get there incrementally by automating verification of one piece of evidence at a time, then incorporating it into the existing change process. We want to avoid abrupt Big Bang rewrites of the organization's change management process.

Action reversibility

Many change management teams want to know about your rollback process. Let's establish some terminology on reversibility.

- Bi-directional changes: The same API call or tool invocation that made the change can also reverse it. Example: Changing tags on an AWS resource with `CreateTags` / `DeleteTags` . Going forward and backward is the same API call but with different parameters.
- Mono-directional changes: Defined as "Get back to where you were but by a different path." Example: An RDS PostgreSQL major version can be reversed by restoring from backups to the previous major version. You can't go straight back to the previous major version.
- Destructive changes: Permanent loss of data or configuration. Example: Destroying a cryptographic key. Sure, you can make a new key but it can never be the same as the old one.

Some changes by their nature are mono-directional or destructive. Your changes will be far easier to justify if you can structure them so that they have bi-directional characteristics, even if there are mono-directional or destructive changes underneath. Example: Deploying blue-green AMI versions so rollback is a single parameter change on the autoscaling group's launch template. The system's production state is genuinely bidirectional even though the machinery underneath isn't.

Change boundaries

When considering a change, the scope of change rarely touches your system alone. Behavioral contracts define the behavior your partner systems can expect, allowing them to determine if your new behavior continues to meet those expectations.

- Full contract verified: The change you're making doesn't violate any contracts with your partner systems (upstream and downstream) and all behavior is defined in them.
- Partial contract: Some of your system's obligations to other systems are covered by contract. Human review is required on the uncovered changes.
- Verbal/inertial contract: Cases where the behavior agreement is verbal or the system's behavior is so old that Grace wrote it. The likelihood of side-effects, even with a fully automated change, can be quite high.
- Unbounded: What are you even planning here?

How it works – bring the receipts

The questions to ask are “Can I reverse my change?” and “Can my partners reverse or tolerate the effects of my change?” These aren't the same question. Detaching an instance profile is trivially reversible (reattach it) but if you do it mid-write, the dropped transactions won't come back when the profile does. The action reverses; the effect doesn't.

If you can say “Yes” to both questions with the receipts to prove it, then your team can use the lighter, machine-verifiable change management process. Destructive changes always require the full review pass even if fully automated. The strongest end of the proof spectrum has these artifacts gathered from lower environments:

App-internal artifacts

As you read through this list of artifacts, consider what format and presentation a change review board would most appreciate. Just because it's easy to dump a megabyte of system logs to a text file attached to a change record doesn't mean the change review board will read it. Easy for you does not equal acceptable to them.

- Automation scripts for rollout and rollback committed to source control.
- Pipelines to execute the scripts that pull from source control.
- API and system audit records of the rollout action. A dump of the raw logs is technically the full and complete change record, but it's not digestible to anyone outside of your team.

- API and system audit records of the rollback/restore action. Establish a legible standardized summary that the change management team can reason about.
- Uptime and stability metrics. Show appropriate application health/throughput metrics before and after the change. Be careful to choose metrics appropriate to your application and that you're willing to support long term.

Partner-system artifacts

Gather these artifacts as appropriate from your partner's systems.

- Contract documents showing support for rollout and rollback operations. Same as audit logs, the raw technical contract specification is unlikely to meet the legibility and traceability needs of the change review board.
- A scheduling conflict view of conflicts with partner or dependent systems. If your compute resources have automated backups, then show how your change is unaffected by competing workload from the backups. Contracts can also describe temporal components that the review board may want to ensure conflict-free changes.
- Uptime and stability metrics of the upstream/downstream applications before and after the change. Everyone loves a pretty graph.
- Logs showing successful processing after rollout and rollback operations.
- Notification records to teams who own the upstream and downstream dependencies. Base these notifications on your behavior contracts for extra visibility.

Incremental advance

Consider at least these two axes when planning out a change management automation effort: What evidence does change management value the most, and what evidence costs the most time for you to generate manually? Aim for this sweet spot to get early wins with the change management team and maximize your time savings.

Limitations and trade-offs

- Technical implementation costs: Shifting to a machine-verifiable change management process will require time, effort, and negotiation on the human and technological control planes. Be iterative and realistic in your expectations.
- Human implementation costs: It's unreasonable to expect a change management team member to understand the deep intricacies of your app's deployment. They are justifiably a conservative bunch. The evidence they've gathered from you previously

has helped them survive tough conversations with auditors. Your evidence has to tell a compelling story months or years from now.

- **Unable to participate:** Some teams cannot qualify for the lightweight review because they cannot generate the audit records or because only manual actions are possible. As much as possible, ensure equitable access for all teams.
- **Organizational flexibility:** Every organization needs to account for the emergency “Make the bleeding stop right now!” situations. You will have won the change management game when your standard change process (with all the attendant evidence) is as fast as a manual emergency change.

Check with your change management team to see if a machine-verifiable change management process is something they’d like to pursue. No doubt they would appreciate a more streamlined process. Remember, the change management team can be considered your customer here. Work with them on what artifact formats and delivery channels will work best for both your teams. Look for a path that gives everyone wins to crow about.



© 2026 FoundryCo, Inc. All Rights Reserved.